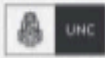


CERTIFICACIÓN UNIVERSITARIA



Manual para el
alumno

Kubernetes I



**WE WANT
SKILLS!**

mundosE

Manual para el alumno: Kubermet I.....	2
Objetivos del encuentro:.....	2
1. Introducción a Kubernetes.....	3
2. Arquitectura y componentes.....	4
3. Pods, Deployments y Services.....	5
4. Manifiestos YAML.....	6
5. kubectl esenciales.....	6
Bibliografía.....	8

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este se presenta un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Kubernetes I

Objetivos del encuentro:

- Explicar el propósito de Kubernetes y su enfoque declarativo.
- Describir la arquitectura de un clúster distinguiendo Control Plane y Data Plane.
- Identificar y caracterizar los objetos básicos: Pod, ReplicaSet, Deployment y Service.
- Comprender la estructura mínima de un manifiesto YAML y las buenas prácticas para versionar.
- Interpretar los comandos esenciales de kubectl para inspección y gestión de recursos.

Contenido a desarrollar:

- ✓ Introducción a Kubernetes
- ✓ Arquitectura y componentes
- ✓ Pods, deployments y services
- ✓ Manifiestos YAML
- ✓ kubectl esenciales

1. Introducción a Kubernetes

Kubernetes, abreviado *k8s*, nació en 2014 como proyecto *open-source* impulsado por Google y luego cedido a la Cloud Native Computing Foundation. Su misión es orquestar contenedores de forma **declarativa**: describes el estado deseado (cuántas réplicas, qué imagen, qué puertos) y el sistema se encarga de alcanzarlo y mantenerlo. Esta filosofía contrasta con enfoques imperativos como scripts Bash donde tienes que enunciar cada paso manualmente.

¿Por qué resulta necesario?

A medida que las aplicaciones se micro-descomponen en decenas o cientos de contenedores, surge la necesidad de *scheduler* (decidir en qué nodo corre cada contenedor), *auto-healing* (reemplazar instancias fallidas), *service discovery* y *rolling updates*. Docker Compose u otras herramientas desarrolladas para entornos pequeños carecen de estas capacidades de manera nativa o, si las ofrecen, no están diseñadas para escalar horizontalmente.

Kubernetes resuelve estos retos con un modelo de **controladores** que observan el estado real y actúan para reconciliarlo con el deseado. El *loop* continuo recuerda a un termostato: mides la temperatura (estado real) y ajustas la caldera (acciones) hasta alcanzar la temperatura deseada. En nuestro caso, si declaras tres réplicas de tu aplicación y una cae, el controlador ReplicaSet lanza automáticamente una nueva.

Este enfoque ganó terreno por cuatro motivos clave:

1. **Portabilidad**: corre en nubes públicas, privadas, bare-metal y edge.
2. **Ecosistema**: cientos de proyectos CNCF extienden sus funcionalidades (observabilidad, seguridad, GitOps).
3. **Comunidad**: miles de colaboradores mantienen código y documentación.
4. **Neutralidad**: no está controlado por un único proveedor.

Empresas como Spotify, Airbnb y CERN lo utilizan para cargas de trabajo que van desde *streaming* de audio hasta análisis de datos científicos. Para el profesional de infraestructura implica adoptar una mentalidad *cloud-native*, enfocada en la automatización y la resiliencia por diseño.

Recursos para profundizar:

- Beda, Joe **[LiveWing]**. (2019, may 30). Keynote - Joe Beda: Kubernetes is a Platform Platform. <https://youtu.be/linfJyjUF10?si=iZ3XPuRSEBP3Ma9K>
- **Burns, B.; Beda, J.; y, Hightower, K.** (2019). Kubernetes: Up & Running. O'Reilly [Capítulos 1 - 2] https://eddiejackson.net/azure/Kubernetes_book.pdf
- **The Kubernetes Authors** © (2025). Documentación de Kubernetes. kubernetes.io/docs/

- **The Linux Foundation®** (2025). Historia de Kubernetes — CNCF (EN).
<https://www.cncf.io/projects/kubernetes/>

2. Arquitectura y componentes

La arquitectura de Kubernetes se divide en **Control Plane** y **Data Plane**. El Control Plane actúa como “cerebro” y está compuesto por:

- **kube-apiserver**: expone la API REST; todos los cambios pasan por este.
- **etcd**: base de datos clave-valor redundante; preserva la fuente de verdad.
- **scheduler**: asigna Pods a nodos basándose en requisitos de CPU, memoria y reglas de afinidad/anti-afinidad.
- **controller-manager**: agrupa controladores (ReplicaSet, Node, Job, etc.) que ejecutan bucles de reconciliación.

En el **Data Plane** cada nodo corre:

- **kubelet**: agente que asegura que los contenedores especificados estén corriendo y saludables.
- **kube-proxy**: maneja reglas iptables/ipvs para exponer los Services.
- **Container Runtime**: containerd o CRI-O, responsable de descargar imágenes y crear contenedores.

Esta separación favorece la escalabilidad: puedes añadir nodos sin tocar la lógica del Control Plane. Para alta disponibilidad se implementan varios *apiservers* más un *etcd* de tres o cinco miembros, de modo que el clúster sigue operativo aun si falla un *master*.

El **scheduler** emplea un algoritmo de “prioridad y selección” que primero filtra nodos incompatibles (por ejemplo, sin suficiente memoria) y luego puntúa los restantes según afinidad o disponibilidad de recursos. El nodo con mejor puntaje recibe el Pod.

Los **controladores** son programas que observan objetos y generan otros para “arreglar” diferencias. Por ejemplo, el ReplicaSet controller compara el número actual de Pods con el deseado; si faltan, crea nuevos; si sobran, elimina. Esta arquitectura de extensibilidad permitió la creación de controladores personalizados (Operators) para gestionar bases de datos o colas de mensajes como primeros ciudadanos del clúster.

Recursos para profundizar:

- **Arilla, C.** (2020). How to monitor Kubernetes control plane. [Blog] Sysdig.
<https://www.sysdig.com/blog/monitor-kubernetes-control-plane>

- **edX** © (2025). LinuxFoundationX: Introduction to Kubernetes [Learn]. <https://www.edx.org/learn/kubernetes/the-linux-foundation-introduction-to-kubernetes>
- **Hernando, V.** (2023). How to Monitor Kubernetes Control Plane with Sysdig. [Blog] Sysdig. <https://www.sysdig.com/blog/how-to-monitor-kubernetes-control-plane>
- **The Kubernetes Authors** © (2025). Componentes de Data Plane. <https://kubernetes.io/es/docs/concepts/overview/components/#componentes-de-nodo>

3. Pods, Deployments y Services

El **Pod** es la unidad mínima de ejecución: comparte red PID y, de ser necesario, volúmenes entre contenedores. Esto posibilita patrones como sidecar (servicios auxiliares) o *init containers* (tareas previas). Por sí solo, un Pod es efímero; al reiniciarse pierde su IP.

Para garantizar disponibilidad se utiliza **ReplicaSet**, que mantiene un número fijo de réplicas. No obstante, trabajar directamente con ReplicaSet limita las actualizaciones. Aquí entra **Deployment**, que abstrae el ReplicaSet y permite **rolling updates**: se reemplazan contenedores gradualmente, controlando ventanas de indisponibilidad con parámetros **maxSurge** y **maxUnavailable**.

Los **Services** solucionan la resolución de nombres y el balanceo L4 dentro del clúster. Hay tres tipos principales:

- **ClusterIP**: accesible solo desde el clúster, ideal para microservicios internos.
- **NodePort**: expone un puerto estático 30k-32767 en cada nodo.
- **LoadBalancer**: integra un balanceador externo del proveedor (ELB, Cloud LB).

El mapeo **Service → Endpoint → Pod IP** se actualiza dinámicamente: si un Pod muere, el controlador Endpoint lo quita del set para evitar tráfico a destinos inactivos.

Por otro lado, **liveness** y **readiness probes** utilizan HTTP, TCP o comandos de *shell* para monitorear salud. Una **readiness** fallida saca al Pod de los Endpoints; una **liveness** fallida lo reinicia. Adoptar probes adecuadas previene errores del tipo *half-open* o contenedores zombis.

Recursos para profundizar:

- **The Kubernetes Authors** © (2025). Configurar las sondas de actividad, preparación y arranque [Configure Liveness, Readiness and Startup Probes]. <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

- **The Kubernetes Authors** © (2025). Container Lifecycle Hooks
<https://kubernetes.io/es/docs/concepts/containers/container-lifecycle-hooks/>
- **Wallarm** (2025). Principales patrones de diseño de Kubernetes.
<https://lab.wallarm.com/what/principales-patrones-de-diseno-de-kubernetes/?lang=es>

4. Manifiestos YAML

YAML —“Yet Another Markup Language”— es el formato que Kubernetes utiliza para codificar el **contrato de estado**. Cada recurso se identifica por **apiVersion**, **kind**, **metadata** y **spec**. Mantener estos documentos versionados en Git permite estrategias **GitOps**: cualquier cambio se propone vía *pull-request* y un controlador (Argo CD, Flux) reconcilia el clúster con la rama *main*.

Una ventaja del modelo declarativo es la **idempotencia**: aplicar el mismo manifiesto varias veces resulta en el mismo estado. Además, Kubernetes almacena la última configuración aplicada (**last-applied-configuration**) y genera parches incrementales, evitando sobrescribir campos tocados por otros procesos.

En cuanto a volúmenes, encontramos tres capas: el **PersistentVolumeClaim** (PVC) que declara la necesidad de almacenamiento, el **PersistentVolume** (PV) que la provee y la **StorageClass** que define los parámetros del backend (tipo de disco, IOPS). De esta forma, las aplicaciones quedan desacopladas de la infraestructura física.

Las configuraciones se externalizan con **ConfigMaps** y **Secrets**. El primero es texto plano; el segundo se codifica en base-64, y se recomienda cifrarlo con KMS o SealedSecrets. Ambos pueden consumirse como variables de entorno o montarse como archivos, permitiendo cambiar parámetros sin reconstruir la imagen.

Errores comunes incluyen sangría inconsistente (produce “mapping values not allowed”), uso de tabs, o modificación de campos inmutables como **selector** en Deployments (provoca recreaciones completas).

Recursos para profundizar:

- **Cheat Sheet oficial YAML** (yaml.org) <https://yaml.org/refcard.html>
- **Rehmanabdul**. (2024). 7 Best Practices for Writing Kubernetes YAML Manifests. Medium [Blog].
<https://blog.devops.dev/7-best-practices-for-writing-kubernetes-yaml-manifests-97554d12b2b9>
- **GitHub** © (2025). Repositorio Git "kubeflow- manifests".
<https://github.com/kubeflow/manifests>

5. kubectl esenciales

`kubectl` es la puerta de entrada al `kube-apiserver`. Internamente, genera una petición HTTP verb `/resource`. Las operaciones se dividen en **lectura** (`get`, `describe`, `logs`) y **mutación** (`apply`, `patch`, `delete`, `scale`).

Con `--field-selector` y `--label-selector` puedes filtrar objetos en tiempo real, mientras que formatos de salida (`-o yaml|json|wide|custom-columns`) facilitan la integración con otras herramientas. Para automatizar, habilita completado Bash/Zsh: `source <(kubectl completion bash)`.

El parámetro `--dry-run=server` valida cambios sin persistir; `kubectl diff` muestra variaciones. Estas dos banderas son vitales en entornos productivos para evitar despliegues accidentales.

`kubeconfig` permite múltiples **contexts**; mantener uno por entorno (`dev`, `stage`, `prod`) y usar alias seguros (`kctx prod && kubectl get ns`) previene errores.

Finalmente, la extensibilidad mediante **plugins** (`krew`) habilita subcomandos como `kubectl neat` (limpia metadata) o `kubectl tree` (muestra jerarquía de recursos).

Recursos para profundizar:

- **The Kubernetes Authors** © (2025). kubectl Cheat Sheet.
<https://kubernetes.io/pt-br/docs/reference/kubectl/cheatsheet/>
- Plugin community index: `krew.dev/plugins/`
<https://github.com/kubernetes-sigs/krew-index>

Bibliografía

Con énfasis en los capítulos introductorios y las guías prácticas:

- Aboullaite, M. (2021). *Learn Kubernetes in a Month of Lunches*, Manning.
- Burns, B.; Beda, J.; y, Hightower, K. (2022). *Kubernetes: Up & Running*. [Capítulos 1 - 2] 3.ª ed., O'Reilly.
- Burns & Grant. (2023). *Kubernetes Patterns*, O'Reilly.
- edX © (2025). LinuxFoundationX: Introduction to Kubernetes [Learn]. <https://www.edx.org/learn/kubernetes/the-linux-foundation-introduction-to-kubernetes>
- Red Hat Developers. (2023). *Pod, Deployment & Service Deep Dive*.
- Tao & Hüttermann. (2022). *GitOps and Kubernetes*, Apress.
- The Kubernetes Authors ©. *Kubernetes Documentation* – Secciones "Concepts". <https://kubernetes.io/docs/home/>